

Retail Sales & Customer Analytics — EDA & Data Cleaning

Objective: Load transaction data from the SQL Server database (built in Phase 1), clean it, engineer new features, and validate key findings from the SQL analysis using Python.

Data source: SQL Server database `RetailAnalytics` (Customers, Products, Orders tables)

Step 1: Import libraries and connect to SQL Server

```
In [11]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sqlalchemy import create_engine
import urllib

server = 'localhost\\SQLEXPRESS'    # update if different
database = 'RetailAnalytics'

params = urllib.parse.quote_plus(
    f'DRIVER={{ODBC Driver 17 for SQL Server}};SERVER={server};DATABASE={database};
')
engine = create_engine(f'mssql+pyodbc:///odbc_connect={params}')

print("Connection engine created")
```

Connection engine created

Step 2: Load data from SQL Server into a DataFrame

Joining Orders, Customers, and Products tables — mirrors the same join logic used in the SQL analysis phase, now pulled into Python for further analysis.

```
In [12]: query = """
SELECT o.RowID, o.OrderID, o.OrderDate, o.ShipDate, o.ShipMode,
       c.CustomerID, c.CustomerName, c.Segment, c.Country, c.City, c.State, c.Posta
       p.ProductID, p.Category, p.SubCategory, p.ProductName,
       o.Sales, o.Quantity, o.Discount, o.Profit
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN Products p ON o.ProductID = p.ProductID
"""

df = pd.read_sql(query, engine)
df.head()
```

Out[12]:

	RowID	OrderID	OrderDate	ShipDate	ShipMode	CustomerID	CustomerName	Segment
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
2	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate
3	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
4	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer

5 rows × 21 columns



Step 3: Initial data quality checks

Checking shape, data types, missing values, and duplicate rows before any cleaning — this establishes a baseline and mirrors the `verify.sql` checks done earlier in the SQL phase.

```
In [13]: print("Shape:", df.shape)
print("\nData types:\n", df.dtypes)
print("\nNull values per column:\n", df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
```

Shape: (9994, 21)

Data types:

RowID	int64
OrderID	object
OrderDate	object
ShipDate	object
ShipMode	object
CustomerID	object
CustomerName	object
Segment	object
Country	object
City	object
State	object
PostalCode	object
Region	object
ProductID	object
Category	object
SubCategory	object
ProductName	object
Sales	float64
Quantity	int64
Discount	float64
Profit	float64

dtype: object

Null values per column:

RowID	0
OrderID	0
OrderDate	0
ShipDate	0
ShipMode	0
CustomerID	0
CustomerName	0
Segment	0
Country	0
City	0
State	0
PostalCode	0
Region	0
ProductID	0
Category	0
SubCategory	0
ProductName	0
Sales	0
Quantity	0
Discount	0
Profit	1

dtype: int64

Duplicate rows: 0

Findings:

- 9,994 rows, 21 columns — matches the SQL `Orders` table row count
- `OrderDate` and `ShipDate` loaded as text (`object`), need conversion to datetime

- `Profit` has 1 null value — matches the known data quality issue identified in SQL (Order ID CA-2017-168389)
- 0 duplicate rows

Step 4: Clean the data

1. Convert `OrderDate` and `ShipDate` to proper datetime format
2. Handle the 1 row with NULL `Profit` — since it's a single row (0.01% of data), it will be dropped and documented rather than imputed, to avoid introducing artificial values into a financial metric

```
In [14]: # Fix date columns
df['OrderDate'] = pd.to_datetime(df['OrderDate'])
df['ShipDate'] = pd.to_datetime(df['ShipDate'])

# Confirm conversion worked
print(df[['OrderDate', 'ShipDate']].dtypes)
print(df[['OrderDate', 'ShipDate']].head())
```

```
OrderDate    datetime64[ns]
ShipDate     datetime64[ns]
dtype: object
   OrderDate  ShipDate
0 2016-11-08 2016-11-11
1 2016-11-08 2016-11-11
2 2016-06-12 2016-06-16
3 2015-10-11 2015-10-18
4 2015-10-11 2015-10-18
```

```
In [15]: # Document the row with NULL Profit before dropping it
null_profit_row = df[df['Profit'].isnull()]
print("Row being dropped due to NULL Profit:")
print(null_profit_row[['OrderID', 'CustomerName', 'Sales', 'Profit']])

# Drop it
df = df.dropna(subset=['Profit'])

print(f"\nRows after cleaning: {df.shape[0]}")
```

```
Row being dropped due to NULL Profit:
   OrderID  CustomerName  Sales  Profit
7344  CA-2017-168389  Darrin Van Huff  721.88    NaN
```

Rows after cleaning: 9993

Step 5: Feature engineering

Creating new columns for deeper analysis:

- `OrderYear`, `OrderMonth` — for time-based trend analysis

- **ShippingDelayDays** — time between order and shipment, a potential operational efficiency metric
- **ProfitMargin** — profit as a percentage of sales, useful for comparing profitability independent of order size

```
In [16]: df['OrderYear'] = df['OrderDate'].dt.year
df['OrderMonth'] = df['OrderDate'].dt.month
df['ShippingDelayDays'] = (df['ShipDate'] - df['OrderDate']).dt.days
df['ProfitMargin'] = df['Profit'] / df['Sales']

df[['OrderYear', 'OrderMonth', 'ShippingDelayDays', 'ProfitMargin']].describe()
```

```
Out[16]:
```

	OrderYear	OrderMonth	ShippingDelayDays	ProfitMargin
count	9993.000000	9993.000000	9993.000000	9993.000000
mean	2015.722105	7.809266	3.957971	0.120377
std	1.123538	3.284551	1.747535	0.466742
min	2014.000000	1.000000	0.000000	-2.759259
25%	2015.000000	5.000000	3.000000	0.074859
50%	2016.000000	9.000000	4.000000	0.270005
75%	2017.000000	11.000000	5.000000	0.362496
max	2017.000000	12.000000	7.000000	0.500220

Findings:

- Dates converted successfully to datetime; no negative shipping delays found (sanity check passed — no order shipped before it was placed)
- Average shipping delay is ~4 days
- Profit margin ranges from -276% to +50% per order. The large gap between mean margin (12%) and median margin (27%) indicates a small number of severely loss-making orders are pulling the average down — consistent with the discount-driven losses identified in the SQL analysis (Query 4)

Step 6: Validate SQL findings using Pandas

Recreating the discount-vs-profit analysis from SQL (Query 4) directly in Pandas, to confirm both approaches produce consistent results.

```
In [17]: def discount_band(d):
    if d == 0:
        return 'No Discount'
    elif d <= 0.2:
        return 'Low (0-20%)'
    elif d <= 0.4:
```

```

        return 'Medium (20-40%)'
    else:
        return 'High (40%+)'

df['DiscountBand'] = df['Discount'].apply(discount_band)

discount_summary = df.groupby('DiscountBand').agg(
    NumOrders=('OrderID', 'count'),
    TotalSales=('Sales', 'sum'),
    TotalProfit=('Profit', 'sum'),
    AvgProfitPerOrder=('Profit', 'mean')
).sort_values('AvgProfitPerOrder', ascending=False)

discount_summary

```

Out[17]:

	NumOrders	TotalSales	TotalProfit	AvgProfitPerOrder
DiscountBand				
No Discount	4798	1087908.47	320987.12	66.900192
Low (0-20%)	3803	846522.13	100785.75	26.501643
Medium (20-40%)	460	234137.96	-35817.57	-77.864283
High (40%+)	932	127910.32	-99138.74	-106.372039

Validation confirmed: The Pandas analysis reproduces the SQL findings almost exactly (minor difference of 1 order due to the dropped NULL-profit row). This confirms:

- Orders with no discount average +\$66.90 profit
- Orders with 40%+ discount average -\$106.37 profit — a net loss

This cross-validation between SQL and Python strengthens confidence in the discount-vs-profit finding as the project's core insight.

Step 7: Save cleaned dataset

Exporting the cleaned, feature-engineered DataFrame to CSV for use in the visualization phase (Matplotlib/Seaborn) and as a Power BI data source.

```

In [18]: df.to_csv('../data/superstore_cleaned.csv', index=False)
print("Saved cleaned dataset:", df.shape)

```

Saved cleaned dataset: (9993, 26)

Step 8: Additional insights (beyond SQL analysis)

Two questions that are easier to explore in Python than SQL:

1. Does shipping delay affect profit or order value?

2. How does profit margin vary by product category — not just total profit, but margin as a % of sales, which reveals hidden loss-makers even in high-revenue categories



Code Cell — Insight 1: Shipping delay vs profit

```
In [19]: shipping_analysis = df.groupby('ShippingDelayDays').agg(  
    NumOrders=('OrderID', 'count'),  
    AvgSales=('Sales', 'mean'),  
    AvgProfit=('Profit', 'mean'),  
    AvgProfitMargin=('ProfitMargin', 'mean')  
) .sort_index()  
  
shipping_analysis
```

```
Out[19]:
```

	NumOrders	AvgSales	AvgProfit	AvgProfitMargin
ShippingDelayDays				
0	519	240.669884	29.645453	0.134544
1	369	184.214932	20.436694	0.131539
2	1334	276.211184	39.818531	0.133350
3	1005	203.641343	26.742119	0.147380
4	2774	227.774730	25.643262	0.098341
5	2169	227.919364	27.078479	0.118341
6	1202	199.308419	28.033328	0.133538
7	621	265.213704	32.740725	0.110413



Code Cell — Insight 2: Profit margin by category

```
In [20]: category_margin = df.groupby('Category').agg(  
    NumOrders=('OrderID', 'count'),  
    TotalSales=('Sales', 'sum'),  
    TotalProfit=('Profit', 'sum'),  
    AvgProfitMargin=('ProfitMargin', 'mean')  
) .sort_values('AvgProfitMargin', ascending=False)  
  
category_margin
```

Out[20]:

	NumOrders	TotalSales	TotalProfit	AvgProfitMargin
Category				
Technology	1847	836154.04	145455.35	0.156144
Office Supplies	6026	719046.93	122490.08	0.138018
Furniture	2120	741277.91	18871.13	0.039075

Findings:

Shipping delay has minimal impact on profitability — average profit margin stays consistently between 10-15% regardless of delivery time (0-7 days), suggesting shipping speed is not a meaningful driver of order profitability in this dataset.

Furniture has a profit margin problem, not a sales problem. Despite generating the second-highest total sales (741,278), *Furniture converts only 3.918,871* — far below Technology (15.6% margin) and Office Supplies (13.8% margin). Total profit figures alone mask this: Furniture looks "fine" in aggregate but is structurally the weakest-performing category on a per-dollar basis.

Business recommendation: Investigate Furniture's cost structure and discounting practices specifically — this category is a strong candidate for the source of the negative-profit orders identified earlier, and may be absorbing disproportionate discounts relative to Technology and Office Supplies.